

Documentación Técnica del Sistema de Simulación de un Convertidor Buck en FPGA

1. Introducción

Este documento describe el funcionamiento interno del proyecto implementado en VHDL para la simulación de un convertidor Buck sobre una FPGA Nexys A7. El sistema genera una señal PWM, calcula numéricamente las variables de estado del convertidor mediante un modelo discreto y transmite dichas variables a dos DAC Pmod DA3 mediante comunicación SPI.

El objetivo principal de este documento es facilitar la comprensión del código fuente para actividades de mantenimiento, depuración y análisis de posibles errores.

2. Arquitectura General

El sistema está compuesto por cinco módulos principales:

- principal
- pwm
- reloj
- buck
- spi

3. Módulo Principal

El módulo principal interconecta todos los bloques funcionales. Sus responsabilidades son:

- Generar la señal PWM.
- Generar la señal tick.
- Ejecutar el modelo Buck.
- Instanciar dos módulos SPI.

Las señales internas más importantes son:

- pwm: controla el interruptor virtual del Buck.
- tick: habilita la actualización del modelo.
- vC: voltaje del capacitor.
- iL: corriente del inductor.

4. Módulo PWM

El módulo PWM convierte un valor de 8 bits en una señal PWM.
El contador interno recorre los valores entre 0 y 1999.
La conversión utilizada es:

$$Duty_{int} = \frac{Duty \times 1999}{255}$$

La frecuencia PWM es:

$$f_{PWM} = \frac{f_{clk}}{2000}$$

Para un reloj de 100 MHz:

$$f_{PWM} = 50kHz$$

5. Módulo Reloj

Este módulo genera una señal de habilitación denominada tick.
Cada vez que el contador alcanza 19 se produce un pulso.

$$f_{tick} = \frac{100MHz}{20} = 5MHz$$

El período asociado es:

$$T_{tick} = 200ns$$

Este período coincide con el valor dt utilizado en el modelo matemático.

6. Modelo Buck(Ya revisado en simulación)

Este módulo implementa un modelo discreto del convertidor Buck utilizando aritmética de punto fijo.

6.1. Parámetros

- $V_{in} = 24V$
- $R = 50\Omega$
- $L = 1mH$

- $C = 100\mu F$
- $dt = 0,2\mu s$

6.2. Corriente del Inductor

Cuando PWM está activo:

$$\frac{di_L}{dt} = \frac{V_{in} - v_C}{L}$$

Cuando PWM está apagado:

$$\frac{di_L}{dt} = \frac{-v_C}{L}$$

La actualización discreta es:

$$i_L(k+1) = i_L(k) + \frac{V_L dt}{L}$$

6.3. Voltaje del Capacitor

$$\frac{dv_C}{dt} = \frac{i_L - \frac{v_C}{R}}{C}$$

Actualización:

$$v_C(k+1) = v_C(k) + \frac{i_C dt}{C}$$

6.4. Método Numérico

Se utiliza integración de Euler hacia adelante:

$$x(k+1) = x(k) + dt\dot{x}(k)$$

7. Módulo SPI

7.1. Objetivo

El módulo SPI tiene la función de transmitir hacia los módulos DAC Pmod DA3 los valores calculados por el modelo Buck. Dentro del sistema existen dos instancias idénticas de este módulo: una dedicada a la transmisión del voltaje del capacitor (v_C) y otra dedicada a la transmisión de la corriente del inductor (i_L).

Cada instancia recibe un valor en formato de punto fijo (**sfixed**) y lo convierte en una trama serial de 16 bits compatible con el DAC.

7.2. Entradas y Salidas

Señal	Tipo	Descripción
clk	std_logic	Reloj utilizado por el módulo
entrada	sfixed(7 downto -24)	Valor que será transmitido al DAC
sclk	std_logic	Reloj SPI generado internamente
cs	std_logic	Señal Chip Select
mosi	std_logic	Línea de transmisión de datos
ldac	std_logic	Señal de actualización del DAC

7.3. Funcionamiento General

El módulo puede dividirse en tres bloques funcionales:

1. Conversión del valor recibido a un número entero de 16 bits.
2. Generación del reloj SPI.
3. Transmisión serial mediante una máquina de estados.

7.4. Conversión de Datos

La entrada del módulo corresponde a un número de punto fijo:

`entrada : sfixed(7 downto -24)`

Antes de transmitirla es necesario convertirla a un valor digital de 16 bits.

El proceso implementado es:

```
if entrada > to_sfixed(2.5,7,-24) then
    temp_int := 65535;
else
    temp_int :=
    to_integer(
    resize(
    (entrada *
    to_sfixed(65535,16,0))
    /
    to_sfixed(25,7,-24),
    32,0));
end if;
```

Cuando la entrada supera el límite definido, el valor transmitido corresponde al máximo posible para un DAC de 16 bits:

65535

En caso contrario se realiza un escalamiento lineal.

La ecuación implementada es:

$$DAC = \frac{entrada}{25} \cdot 65535$$

Esta operación transforma la variable física calculada por el modelo Buck en un valor digital que posteriormente será enviado al DAC.

7.5. Generación del Reloj SPI

El reloj SPI es generado internamente mediante un contador.

```
if counter = 49 then
    counter <= 0;
    spi_clk <= not spi_clk;
else
    counter <= counter + 1;
end if;
```

Cada vez que el contador alcanza el valor 49 el reloj cambia de estado. Como consecuencia, un período completo requiere 100 ciclos de reloj. La frecuencia obtenida es:

$$f_{SPI} = \frac{f_{clk}}{100}$$

Dentro de este diseño el módulo recibe como reloj la señal `tick`. Dado que:

$$f_{tick} = 5MHz$$

la frecuencia final de transmisión es:

$$f_{SPI} = 50kHz$$

7.6. Máquina de Estados

La transmisión se controla mediante una máquina de estados sencilla.

```
type tx_state_type is
(
    idle,
    transmitting
);
```

7.6.1. Estado Idle

En este estado se prepara una nueva transmisión.

```
shift_reg <= std_logic_vector(dac_data);
bit_count <= 16;
tx_state <= transmitting;
```

Las acciones realizadas son:

- Cargar el dato a transmitir.
- Reiniciar el contador de bits.
- Iniciar una nueva transmisión.

7.6.2. Estado Transmitting

Durante este estado se envían secuencialmente los 16 bits de la palabra digital. La transmisión ocurre en cada flanco descendente del reloj SPI.

```
if sck_previous='1'
and spi_clk='0' then
```

7.7. Detección de Flancos

Para detectar el flanco descendente se almacena el estado previo del reloj.

```
sck_previous <= spi_clk;
```

Posteriormente se compara con el valor actual:

```
if sck_previous='1'
and spi_clk='0'
```

Esta técnica permite detectar exactamente cuándo el reloj cambia de nivel alto a nivel bajo.

7.8. Transmisión de Datos

El dato transmitido por MOSI corresponde inicialmente al bit más significativo.

```
mosi_int <= shift_reg(15);
```

Esto significa que la transmisión se realiza en formato MSB First.

Después de transmitir cada bit se desplaza el contenido del registro:

```
shift_reg <=
shift_reg(14 downto 0) & '0';
```

Como resultado:

- Primero se transmite el bit 15.
- Luego el bit 14.
- Después el bit 13.
- Y así sucesivamente hasta el bit 0.

7.9. Finalización de la Trama

La variable `bit_count` controla cuántos bits faltan por transmitir.

Cuando únicamente queda un bit:

```
if bit_count = 1 then
```

la transmisión finaliza y la máquina regresa al estado `idle`.

```
tx_state <= idle;
cs_int <= '1';
```

Posteriormente se prepara automáticamente una nueva palabra para ser enviada.

7.10. Máquina de Estados

Estados:

- idle
- transmitting

Durante idle se carga el registro de desplazamiento.

Durante transmitting se envían los 16 bits por MOSI.

7.11. Frecuencia SPI

El reloj SPI se obtiene dividiendo la frecuencia de tick.

$$f_{SPI} = \frac{f_{tick}}{100}$$

Por tanto:

$$f_{SPI} = 50kHz$$

8. Asignación de Pines

8.1. Entradas

Señal	Pin
duty[7]	R13
duty[6]	U18
duty[5]	T18
duty[4]	R17
duty[3]	R15
duty[2]	M13
duty[1]	L16
duty[0]	J15
rst	N17
clk	E3

8.2. DAC Voltaje

Señal	Pin
cs_V	C17
mosi_V	D18
ldac_V	E18
sclk_V	G17

8.3. DAC Corriente

Señal	Pin
cs_I	D14
mosi_I	F16
ldac_I	G16
sclk_I	H14